

# CLEF: A Python Framework for Closed-Loop Neuroscience Experiments

Raymond L. Dunn<sup>1</sup> and Saul Kato<sup>1</sup>

<sup>1</sup> Department of Neurology, Weill Institute for Neurosciences, University of California, San Francisco, CA, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: ¶

Submitted: 05 June 2026

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

CLEF (Closed-Loop Experimental Framework) is an open-source Python platform for real-time closed-loop experiments with automated stimulus control. This first CLEF release targets microscopic imaging modalities for neuroscience researchers. CLEF enables researchers to observe neural activity or organism behavior, process real-time data according to pre-defined user-specified algorithms, and deliver stimuli in a single feedback loop governed entirely by declarative configuration files. CLEF also provides a GUI interface for live monitoring and controlling experiments.

The core philosophy behind CLEF is that a flexible framework will allow rapid design of custom experiments that naturally scale. CLEF replaces ad-hoc lab scripts and GUI-only workflows with a modular, config-driven architecture where hardware backends, real-time analysis algorithms, and stimulus controllers are interchangeable components selected at runtime via YAML. Think of it as an agentic toolkit for real-world experiments.

## Statement of Need

### Closed-Loop Experimentation for Advancing Neuroscience

Traditional neuroscience experiments typically follow a fixed, open-loop protocol: researchers design the experiment, set parameters, collect data subject to a fixed stimulus protocol, and analyze results afterward. This approach has yielded insight for many questions, but it is fundamentally impoverished approach to how dynamical biological systems operate. Brains are highly recurrent networks where activity patterns influence future states in complex ways. To understand causal relationships in these systems, we need experiments that can perturbatively and adaptively probe a system, responsive to the evolving system state itself.

Closed-loop experimental design allows real-time adaptation of stimulus protocols based on ongoing measurements. The value of this approach has been recognized across multiple areas of study and model systems (Grosenick et al., 2015). As measurement technologies scale to capture hundreds or thousands of neurons simultaneously, closed-loop methods become increasingly important for understanding network-level mechanisms of brain function.

### The Need for Flexible Automation Tools

Closed-loop experiments require coordinated control of multiple hardware components (cameras, stages, stimulation devices) while performing low-latency, real-time computation on streaming data. The computational pipeline must extract features from raw measurements, decide what to do based on the current state of the interrogated system, and execute stimulus protocols

with precise timing. Building this coordination layer from scratch for every new experiment is a barrier to wider adoption of closed-loop methods.

Existing tools cover individual parts of the problem. While Python is the predominant programming language in the biosciences, many established frameworks are written in non-Python languages: Bonsai in C# (Lopes et al., 2015), the Open Ephys GUI in C++ (Siegle et al., 2017), and ScanImage in MATLAB (Pologruto et al., 2003). Several otherwise-Python tools target a different domain or stage of the workflow: Open Ephys focuses on electrophysiology rather than imaging (Siegle et al., 2017), while CalmAn (Giovannucci et al., 2019) and Suite2p (Pachitariu et al., 2016) provide offline calcium-imaging analysis without experiment orchestration. Tools such as Stytra (Štih et al., 2019), ACQ4 (Campagnola et al., 2014), and Autopilot (Saunders et al., 2019) provide closed-loop control but are specialized to particular rigs or behavioral paradigms. Commercial microscopy software exposes limited scripting that works for simple automated protocols but lacks the sophistication needed for responsive, state-dependent experiments. Pycro-Manager (Pinkard et al., 2021) established convenient Python-based microscope control on top of Micro-Manager (Edelstein et al., 2014) and enabled more complex acquisition sequences, but it does not, on its own, supply the architecture needed for closed-loop experimentation with multiple hardware components and real-time decision-making outside of the Micro-Manager ecosystem. The CLEF framework is written entirely in Python, so it is accessible to the large community of scientists already using Python. Researchers can use NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), scikit-learn (Pedregosa et al., 2011), and PyTorch (Paszke et al., 2019) directly inside their experimental logic, as well as any other code libraries.

## Architecture

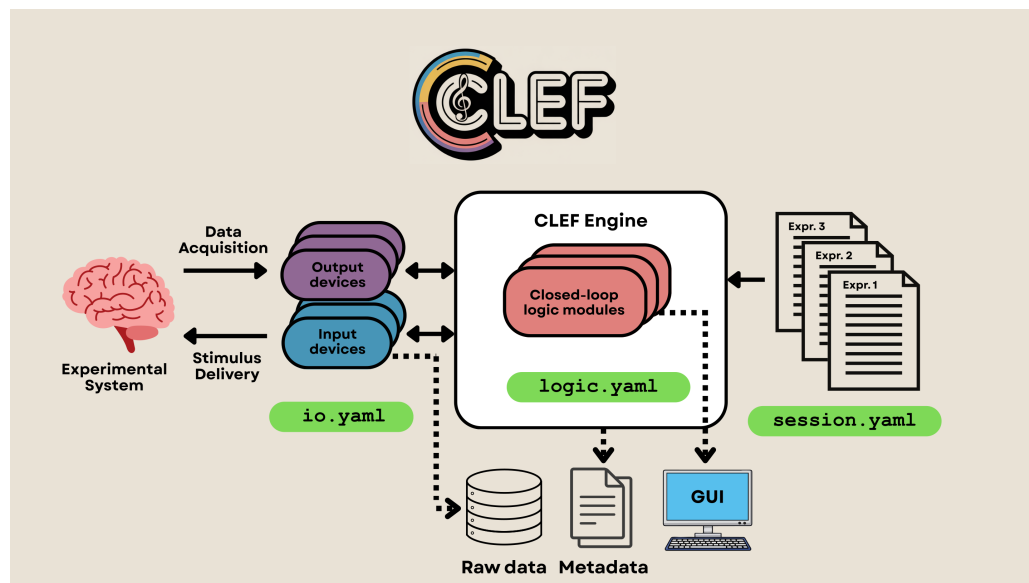
CLEF uses a layered architecture in which independent components communicate through simple interfaces. Researchers can extend parts of the system (adding new hardware, implementing new logic algorithms) for their individual application without needing to modify the rest of the codebase.

## Key Concepts

CLEF is organized around four concepts used throughout the rest of this manuscript:

| Concept | Purpose                                                                                                                                                                                                                                                     |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| device  | Components or endpoints that CLEF interacts with. A device is either an input_device or an output_device. Input devices provide data streams (e.g. camera images), and output devices are what the experimenter wants to control (e.g. a stage or a laser). |
| logic   | A control algorithm. It processes samples from the input devices and emits updates for the output devices.                                                                                                                                                  |
| session | Contextual metadata for an experiment. For example, information about the data subject (cell line, treatment condition), highly specific to the application.                                                                                                |
| engine  | A discrete event loop that orchestrates iterations of data sampling, data processing, and actuation.                                                                                                                                                        |

An experiment can use one or more input devices and one or more output devices: multiple cameras, a camera plus a stage readout, or a DMD (digital micromirror device) plus a laser plus a stage. At runtime, the engine reads from every registered input device, passes the samples to the logic algorithm, and dispatches output commands to any subset of the registered output devices (Figure 1).



**Figure 1:** System overview of CLEF. Three YAML configuration files (`io.yaml`, `logic.yaml`, `session.yaml`) are validated by Pydantic models and passed to the ClosedLoopEngine, which orchestrates the real-time acquisition loop. The engine reads samples from one or more registered `input_devices`, passes data samples to the active logic algorithms, and dispatches return values to one or more named `output_devices`. The engine collects per-device and per-algorithm metadata generated during a session.

## Code Architecture

The repository is split into two top-level trees. `core/` contains the framework internals: base classes, managers, the engine, configuration logic, and the CLI. `apps/` contains concrete implementations that register against the core base classes via plugin discovery, plus per-application YAML configs.

- **CLI entry point** (`core/utis/clef_cli.py`) accepts either a preset name (e.g. `clef <app_name>`) that resolves to a directory under `apps/config/`, or explicit `--session`, `--io`, and `--logic` paths.
- **ClosedLoopEngine** (`core/engine/closed_loop_engine.py`) owns an `IOManager` and a `LogicManager` and runs the sample loop: acquire from input devices, pass to logic, dispatch any returned commands to output devices.
- **ConfigManager** (`core/config/config_manager.py`) loads, validates, and merges the three configuration files against Pydantic models (`SessionConfig`, `IIOConfig`, `ClosedLoopLogicConfig`), with defaults in `core/config/defaults/`. Validation catches invalid parameters before any hardware is initialized.
- **IOManager** (`core/io/`) instantiates `BaseInputDevice` and `BaseOutputDevice` subclasses listed in `io.yaml`. Each input device is paired with a `BaseDataInterface` that controls how the data stream is buffered, structured, and persisted at the end of the session.
- **LogicManager** (`core/logic/`) instantiates the `BaseClosedLoopLogic` subclass selected by `logic.yaml` and connects it to the input/output devices held by the `IOManager`.

At runtime, the data flow is symmetric and minimal:

`input_devices` → engine → closed-loop logic → engine → `output_devices`

## Customizing CLEF for your New Experiment

A new closed-loop experiment is built by writing (or reusing) one or more input devices, one or more output devices, one or more logic algorithms, and three configuration files. The four base abstractions (`BaseInputDevice`, `BaseOutputDevice`, `BaseDataInterface`, and `BaseClosedLoopLogic`) each define a small interface that researchers fill in for their own hardware and analysis. A single experiment can fan in samples from several cameras, electrode signals, or stage readouts, and fan out to multiple actuators (a DMD plus a light source plus a stage, for example). The customization workflow follows these steps:

1. **Create an input device.** Subclass `BaseInputDevice` to wrap any source of streaming data (a camera, an electrode amplifier, a stage readout, a TIFF stack), implementing `connect()`, `configure()`, `_get_input()`, and `close()`.
2. **Create an output device.** Subclass `BaseOutputDevice` to wrap any actuator (an LED, a DMD, a motorized stage), implementing `connect()`, `configure()`, `_update_output(**kwargs)`, and `close()`.
3. **Create a closed-loop logic algorithm.** Subclass `BaseClosedLoopLogic` to implement the experiment's online analysis and decision rules via `initialize_model()`, `process_sample(sample)`, `_check_logic()`, and `close()`.
4. **Pair input devices with a data interface (optional).** Subclass `BaseDataInterface` to separate *how a sample looks* from *how it is stored*, defining sample shape, dtype, and end-of-session serialization. New modalities can be added with no engine changes, and the per-session output bundle maps cleanly onto storage systems such as `minimo` (Borchardt et al., 2021).
5. **Write the three configuration files.** `io.yaml` lists the input and output devices and their parameters; `logic.yaml` selects the algorithm and its tunable parameters; `session.yaml` describes the run itself (operator, subject, conditions, output location, duration). All three are validated by Pydantic at startup, before any hardware is initialized.
6. **Run the experiment.** The CLI shorthand `clef <app_name>` resolves to `apps/config/<app_name>/` and loads all three files. At the end of every session, CLEF writes per-device data files plus a single JSON metadata file capturing the merged configuration, every output event with its timestamp, and per-device sample timestamps.

Full descriptions of each base class's interface, the registry mechanism, configuration schemas, and the CLI are provided in the online documentation.

## Demos

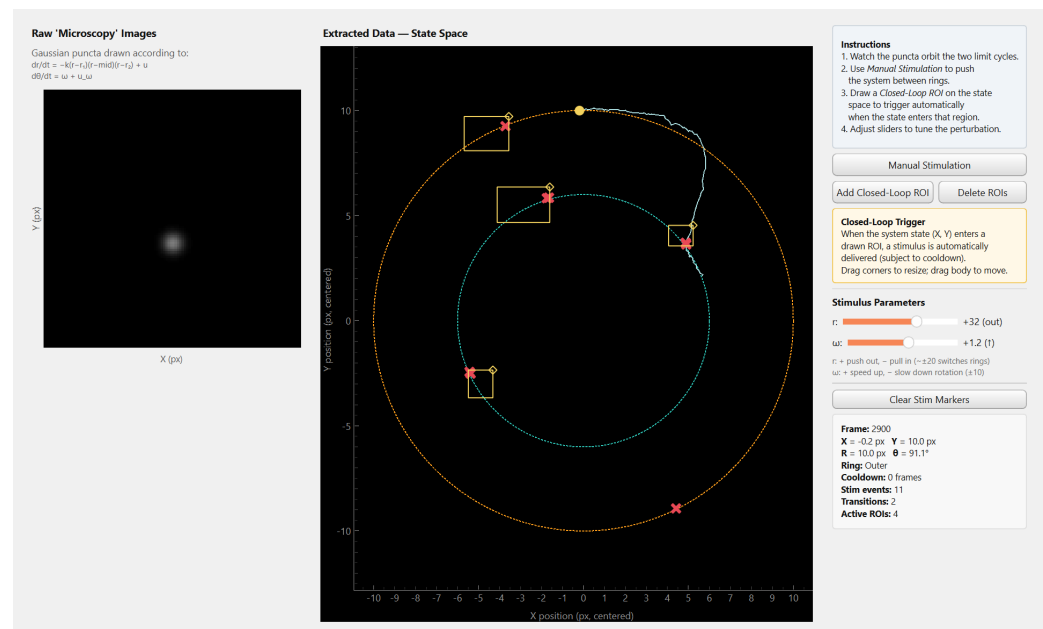
CLEF ships with several demo configurations covering a range of use cases, including synthetic dynamical systems, recorded data playback, and real microscope hardware. Each demo lives in its own directory under `apps/config/`.

### Limit Cycle Demo (`apps/config/limit_cycle/`)

The limit cycle demo captures the core conceptual motivation for CLEF. Closed-loop experimental frameworks exist to interrogate dynamical biological processes, and brains are perhaps the most compelling example. In this demo, a virtual live dynamical system (a bistable system with two concentric limit cycles) produces time-varying activity, and the experimenter delivers perturbations to understand how the system works. The setup is essentially a game: the system evolves according to its own hidden dynamics, and the experimenter must observe, hypothesize, and intervene in real time to discern how these dynamics are structured.

The `limit_cycle_input` device generates 2-D image frames showing a punctum orbiting on one of two concentric rings (Figure 2). The system state is defined by an angular position and a radial mode (inner or outer ring). The `limit_cycle_output` device delivers stimuli that

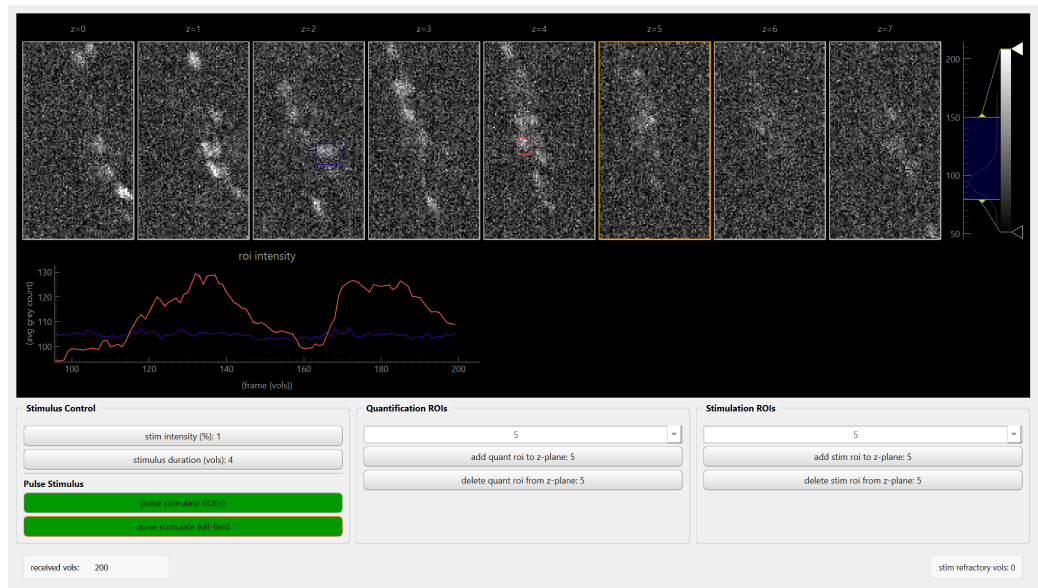
141 perturb the angular position or toggle the system between rings. An auto-trigger mode is  
142 available, in which the logic fires stimuli when the system enters a specified angular region.  
143 The demo illustrates how CLEF separates data generation, online analysis, and stimulus control  
144 into independently configurable components.



**Figure 2:** Screenshot of the limit cycle demo during a live session. The visualization shows the punctum orbiting on one of two concentric rings, with a fading trajectory trail indicating recent history. The experimenter can deliver angular perturbations or toggle the system between rings using the GUI controls, observing the effect of each intervention in real time.

## 145 Recording Playback Demo (apps/config/recording\_playback/)

146 The recording playback demo provides the same GUI and analysis interface as the physical  
147 hardware configuration, but reads data from an existing volumetric calcium imaging dataset  
148 (a TIFF stack) via recording\_playback\_input rather than acquiring live from a microscope.  
149 Users can develop, test, and refine their analysis pipelines against real neural data without  
150 needing access to microscope hardware. The brainalyzer\_logic algorithm performs real-time  
151 quantification of neural activity across z-planes (offloaded to brainalyzer\_worker.py over  
152 shared memory) and supports stimulus parameter exploration through the interactive GUI  
153 (Figure 3). This demo illustrates how application-specific workflows can be integrated and  
154 tested with CLEF prior to deployment.



**Figure 3:** Screenshot of the “brainalyzer” pipeline running over the recording-playback demo. The interface displays volumetric calcium imaging data read from a TIFF stack, with real-time quantification of neural activity across z-planes. This demo provides the same analysis and stimulus control interface as the physical hardware configuration, allowing algorithm development and parameter exploration without a connected microscope.

### Speech BCI Demo (apps/config/speech\_bci/)

The speech BCI demo illustrates the network-service logic pattern and also is an example of CLEF for 1-D timeseries data (electrophysiology of spiking neurons). `speech_bci_input` provides neural spiking data, the logic algorithm streams it over WebSocket to a FastAPI decoder server (`apps/subprocess/speech_bci_server.py`) that runs a GRU + n-gram language model inside Docker (locally or on a GCP Compute Engine VM via `docker/deploy_decoder.sh`), and the decoded text is rendered through `speech_bci_output`. Alternatively, for minimizing latency introduced by inter-process communication on systems where model subprocesses are hosted on the same hardware as data streams, zero-copy data transfer via shared-memory buffers is also implemented. The demo shows how CLEF supports workflows with deployed, standalone, networked models.

### Physical Hardware Demo (apps/config/physical\_hardware/)

The physical hardware demo runs a full volumetric calcium imaging experiment with real-time quantification and patterned optogenetic illumination using the Mightex Polygon1000 digital micromirror device. It can only be run with the appropriate microscope hardware configured (camera, stage, Micro-Manager device adapters, and Polygon1000, 89North LDI LS). This is the primary production use case for CLEF: acquiring volumetric calcium imaging data, processing it online via the brainalyzer logic, and delivering spatially patterned optogenetic stimuli in a closed loop. We include this demo as an illustration of CLEF being applied daily in our own lab (Dunn et al., 2025).

## Availability and Documentation

CLEF is open-source software released under the MIT license. The code is available on GitHub at <https://github.com/focolab/clef>. Documentation includes installation instructions,



178 configuration guides, a vibe-coding quickstart for AI-assisted app development, and the demo  
179 configurations described above.

180 We welcome contributions from the community. The modular architecture supports adding  
181 new hardware devices or new logic types without modifying framework code. Issues and pull  
182 requests can be opened on the GitHub repository.

## 183 AI Usage Statement

184 AI coding assistants were used during development to facilitate refactoring of parts of the  
185 codebase, improve test coverage, and to facilitate manuscript citation formatting. All scientific  
186 content, architectural decisions, and final code were authored and reviewed by the listed  
187 authors.

## 188 Acknowledgements

189 We thank members of the Kato lab for valuable discussion.

190 Data for this study were acquired at the UCSF Innovation Core at the Weill Institute for  
191 Neurosciences on a custom imaging system controlled by the open-source software packages  
192 Micro-Manager and Pycro-Manager.

193 This work was supported by NIH grants NS115572 (R.L.D.), R35GM124735 (S.K.), and the  
194 Weill Institute for Neurosciences (S.K.).

## 195 References

- 196 Borchardt, J. M., Dunn, R., & Kato, S. (2021). Minimo: A linked data and metadata  
197 storage system for small labs. *Journal of Open Source Software*, 6(62), 2979. <https://doi.org/10.21105/joss.02979>  
198
- 199 Campagnola, L., Kratz, M. B., & Manis, P. B. (2014). ACQ4: An open-source software platform  
200 for data acquisition and analysis in neurophysiology research. *Frontiers in Neuroinformatics*,  
201 8, 3. <https://doi.org/10.3389/fninf.2014.00003>
- 202 Dunn, R. L., Costello, C. M., Borchardt, J. M., Sprague, D. Y., Chiu, G. C., Miller, J. M.,  
203 L'Etoile, N. D., & Kato, S. (2025). Short-term memory by distributed neural network  
204 oscillators in a simple nervous system. *Current Biology*, 35(22), 5582–5593.e4. <https://doi.org/10.1016/j.cub.2025.10.018>  
205
- 206 Edelstein, A. D., Tsuchida, M. A., Amodaj, N., Pinkard, H., Vale, R. D., & Stuurman, N.  
207 (2014). Advanced methods of microscope control using  $\mu$ manager software. *Journal of*  
208 *Biological Methods*, 1(2), e10. <https://doi.org/10.14440/jbm.2014.36>
- 209 Giovannucci, A., Friedrich, J., Gunn, P., Kalfon, J., Brown, B. L., Koay, S. A., Taxidis, J.,  
210 Najafi, F., Gauthier, J. L., Zhou, P., Khakh, B. S., Tank, D. W., Chklovskii, D. B., &  
211 Pnevmatikakis, E. A. (2019). CalmAn an open source tool for scalable calcium imaging  
212 data analysis. *eLife*, 8, e38173. <https://doi.org/10.7554/eLife.38173>
- 213 Grosenick, L., Marshel, J. H., & Deisseroth, K. (2015). Closed-loop and activity-guided  
214 optogenetic control. *Neuron*, 86(1), 106–139. <https://doi.org/10.1016/j.neuron.2015.03.034>  
215
- 216 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,  
217 Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,  
218 M. H. van, Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., ...

- 219 Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.  
220 <https://doi.org/10.1038/s41586-020-2649-2>
- 221 Lopes, G., Bonacchi, N., Frazão, J., Neto, J. P., Atallah, B. V., Soares, S., Moreira, L., Matias,  
222 S., Itskov, P. M., Correia, P. A., Medina, R. E., Calcaterra, L., Dreosti, E., Paton, J. J., &  
223 Kampff, A. R. (2015). Bonsai: An event-based framework for processing and controlling data  
224 streams. *Frontiers in Neuroinformatics*, 9, 7. <https://doi.org/10.3389/fninf.2015.00007>
- 225 Pachitariu, M., Stringer, C., Dipoppa, M., Schröder, S., Rossi, L. F., Dalgleish, H., Carandini,  
226 M., & Harris, K. D. (2016). Suite2p: Beyond 10,000 neurons with standard two-photon  
227 microscopy. *bioRxiv*. <https://doi.org/10.1101/061507>
- 228 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z.,  
229 Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M.,  
230 Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An  
231 imperative style, high-performance deep learning library. *Advances in Neural Information  
232 Processing Systems* 32, 8024–8035.
- 233 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,  
234 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,  
235 Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python.  
236 *Journal of Machine Learning Research*, 12, 2825–2830.
- 237 Pinkard, H., Stuurman, N., Ivanov, I. E., Anthony, N. M., Ouyang, W., Li, B., Yang, B.,  
238 Tsuchida, M. A., Chhun, B., Zhang, G., Mei, R., Anderson, M., Shepherd, D. P., Hunt-  
239 Isaak, I., Dunn, R. L., Jahr, W., Kato, S., Royer, L. A., Thiagarajah, J. R., ... Waller, L.  
240 (2021). Pycro-manager: Open-source software for customized and reproducible microscope  
241 control. *Nature Methods*, 18(3), 226–228. <https://doi.org/10.1038/s41592-021-01087-6>
- 242 Pologruto, T. A., Sabatini, B. L., & Svoboda, K. (2003). ScanImage: Flexible software  
243 for operating laser scanning microscopes. *BioMedical Engineering OnLine*, 2(1), 13.  
244 <https://doi.org/10.1186/1475-925X-2-13>
- 245 Saunders, J. L., Ott, L. A., & Wehr, M. (2019). Autopilot: Automating experiments with lots  
246 of raspberry pis. *bioRxiv*. <https://doi.org/10.1101/807693>
- 247 Siegle, J. H., López, A. C., Patel, Y. A., Abramov, K., Ohayon, S., & Voigts, J. (2017). Open  
248 ephys: An open-source, plugin-based platform for multichannel electrophysiology. *Journal  
249 of Neural Engineering*, 14(4), 045003. <https://doi.org/10.1088/1741-2552/aa5eea>
- 250 Štíh, V., Petrucco, L., Kist, A. M., & Portugues, R. (2019). Stytra: An open-source,  
251 integrated system for stimulation, tracking and closed-loop behavioral experiments. *PLOS  
252 Computational Biology*, 15(4), e1006699. <https://doi.org/10.1371/journal.pcbi.1006699>
- 253 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,  
254 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,  
255 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E.,  
256 ... Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing  
257 in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>